1

1.1 死亡人数靠前的国家及总死亡人数

```
#1、1
country=Sig_Eqs.groupby(['Country'])['Deaths'].sum().sort_values(ascending=False)
country.head(20)
```

```
Country
CHINA           2139209.0
TURKEY          1199720.0
IRAN            1014453.0
ITALY            492236.0
SYRIA            419826.0
HAITI            323484.0
AZERBAIJAN       319251.0
JAPAN            242516.0
ARMENIA          177890.0
PAKISTAN         145085.0
IRAQ             136200.0
ECUADOR          135496.0
TURKMENISTAN     117412.0
PERU             102309.0
ISRAEL            93388.0
PORTUGAL          83572.0
GREECE            81162.0
CHILE             64277.0
INDIA             63507.0
TAIWAN            57154.0
Name: Deaths, dtype: float64
```

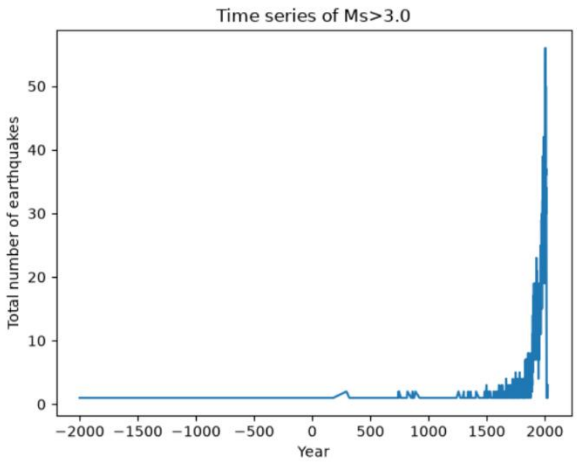
1.2

1.2.1 思路：

一开始我使用“total_number=Sig_Eqs.loc[Sig_Eqs['Ms']>3.0].groupby(['Year']).plot(kind='line')”结果 jupyter notebook 直接卡死，并弹出 Runtime warning 的警告，之后我尝试在 groupby 和 plot 之间加入 count()，结果图上包含所有的列数据，最后我提取出 Ms 为指定列进行计数，成功输出。

1.2.2 绘制时间序列：

```
[8]: #1、2
total_number=Sig_Eqs.loc[Sig_Eqs['Ms']>3.0].groupby(['Year'])['Ms'].count().plot(kind='line')
plt.xlabel('Year')
plt.ylabel('Total number of earthquakes')
plt.title('Time series of Ms>3.0')
plt.show()
```



1.2.3 趋势分析：

我们发现，在 1900 年之前，震级大于 3.0 的地震发生的次数很少，约 1900 年之后，震级大于 3.0 的地震发生的次数急剧增长。可能是由于 1900 年之前大家对地震没有很高的认识，对此的记录不够全面，而随着科技的发展与地震监测技术的进步，1900 年后的仪器可以监测并到 1900 年之前无法被感知与记录的地震，使得监测数量急剧增长。

1.3

1.3.1 思路

一开始我用 for 循环遍历未去重的国家，尝试用 Series 拼装，结果直接卡死并报错；之后改用 DataFrame 和 concat 拼接，因未加 unique()去重，造成重复计算和无限重排，再次导致电脑卡死；最后我通过在获取国家名单时加上 unique()去重，并正确打印 result.head(10)`，最终成功输出结果。

注：

在报错后，通过谷歌检索 pandas 清洗数据的方法，学习 dropna()可以用来删除缺失值，unique()可以去除重复值；用 series 拼装报错后，我通过谷歌检索 pandas 数据处理方法，学习了如何用 DataFrame 构建空表以及用 pd.concat 进行数据拼接。

```
result=pd.concat([result, new]).reset_index(drop=True).sort_values(by='Total_number', ascending
result.head(10)
```

3]:

	Country	Total_number	Year	Location
0	CHINA	626	1920.0	CHINA: GANSU PROVINCE, SHANXI PROVINCE
1	JAPAN	429	869.0	JAPAN: SANRIKU
2	INDONESIA	427	2004.0	INDONESIA: SUMATRA: ACEH: OFF WEST COAST
3	IRAN	385	856.0	IRAN: DAMGHAN, QUMIS
4	TURKEY	359	1999.0	TURKEY: ISTANBUL, KOCAELI, SAKARYA
5	ITALY	338	1915.0	ITALY: MARSICA, AVEZZANO, ABRUZZI
6	GREECE	296	365.0	GREECE: CRETE: KNOSSOS
7	USA	283	1957.0	ALASKA
8	PHILIPPINES	233	1897.0	PHILIPPINES: MINDANAO, ZAMBOANGA, SULU, ISABELA
9	MEXICO	217	1787.0	MEXICO: SAN MARCOS, OAXACA

]:

]:

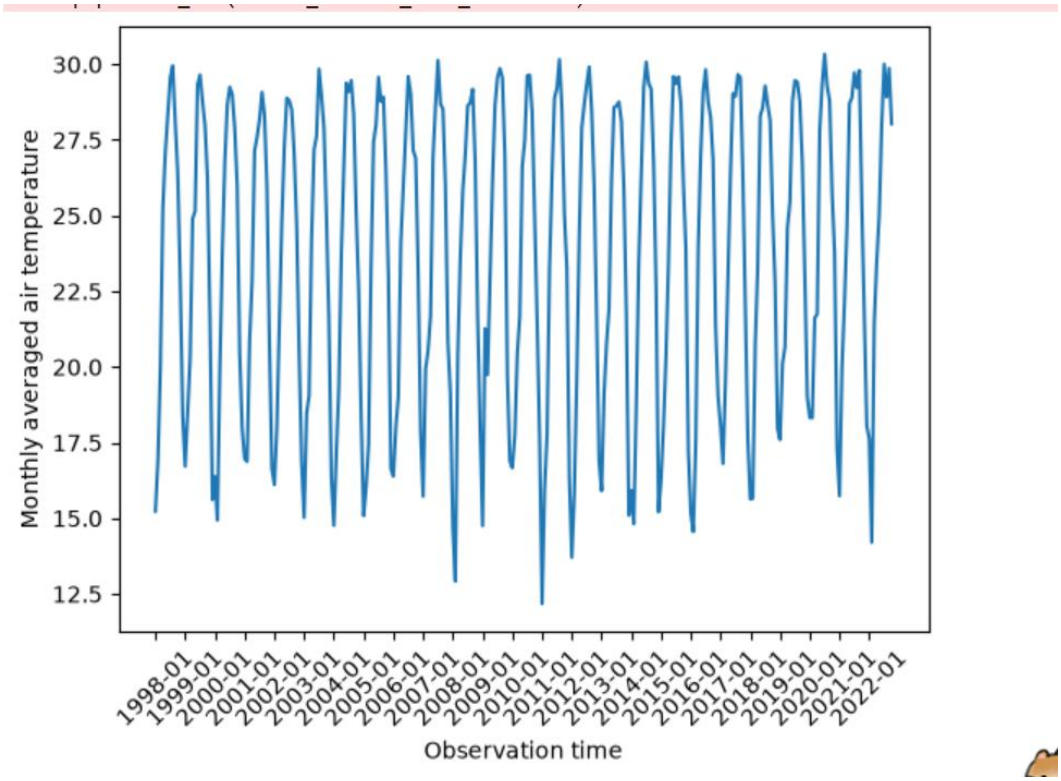
2

2.1 思路

- 阅读 pdf 了解数据格式
- 利用 pd.read_csv()读取数据
- 清洗数据：去除缺失值+99999，根据手册说明的最小/最大限制利用布尔索引筛选有效值
- 提取有效数据：利用 str[0:5]提取前五位数值，用 astype(float)转换格式，再/10 得到摄氏度
- 时间提取：使用 str[0:7] 从日期时间中提取出年月
- 分组计算月平均值：利用 groupby('month')['TMP'].mean()
- 绘图

注：在得出的图像中 x 轴的数据挤成一团，通过浏览器检索 matplotlib 绘图技巧，学习如何显示部分标签，如何使得标签旋转，习得 plt.xticks()的应用

2.2 绘制月平均气温与观测时间的关系图



2.3 1998 年至 2022 年的月平均气温有何趋势？

据图可知，1998 年至 2022 年深圳宝安的月平均气温呈现出规律的周期性季节波动（夏季高、冬季低），年最高温与最低温无明显差异，2007-1 与 2010-1 的温度为历年偏低气温。

3

3.1 按风暴识别码（SID）对数据进行分组，根据风速（WMO_WIND）列出 10 个最强飓风的名称（NAME）。

按照 SID 和 NAME 进行分组，找出每组的最大风速，然后降序排序取前 10。（groupby()和 sort_values()。

```
[10]: SID            NAME
      1950224N17305  ABLE       95
      1950234N11334  CHARLIE    95
      1950244N20277  EASY       95
      1950251N16319  FOX        95
      1950270N25309  GEORGE     95
      1951224N12316  CHARLIE    95
      1951244N13328  EASY       95
      1951246N13340  FOX        95
      1952244N15306  BAKER      95
      1952268N20291  CHARLIE    95
      Name: WMO_WIND, dtype: str
```

3.2 列出 20 个最强飓风的风速（WMO_WIND）。

对 WMO_WIND 这一列降序排序，展示前 20 sort_values(ascending=False).head(20)

```
13]: 46787    95
     46789    95
     46791    95
     46815    95
     46817    95
     46819    95
     46821    95
     46823    95
     46942    95
     46952    95
     47532    95
     49333    95
     49375    95
     49450    95
     49466    95
     52446    95
     52448    95
     52450    95
     52452    95
     52454    95
     Name: WMO_WIND, dtype: str
```

3.3 按流域统计所有数据点数量。

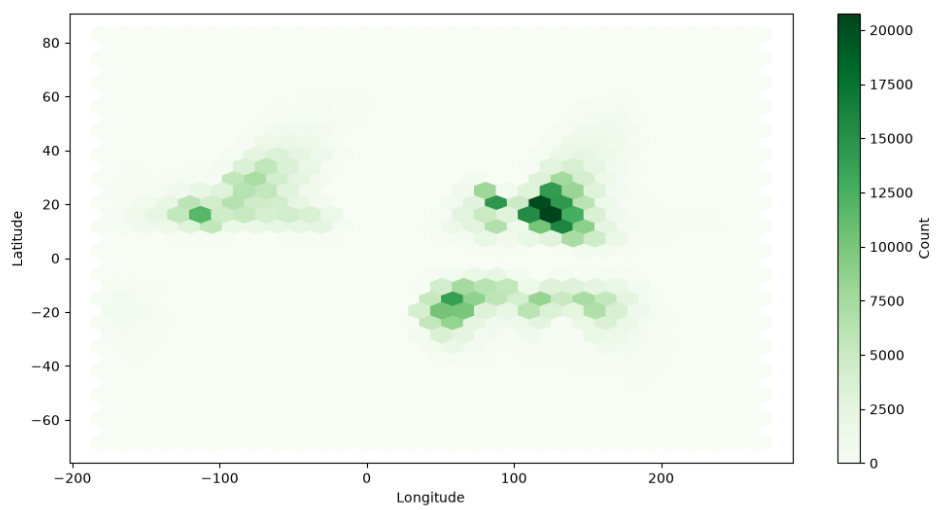
按 BASIN 列分组再用 counts()计数

```
15]:  BASIN
      EP     62412
      NI     55401
      SA       119
      SI    160668
      SP     67119
      WP    236576
      Name: BASIN, dtype: int64
```

```
[ ]:
```

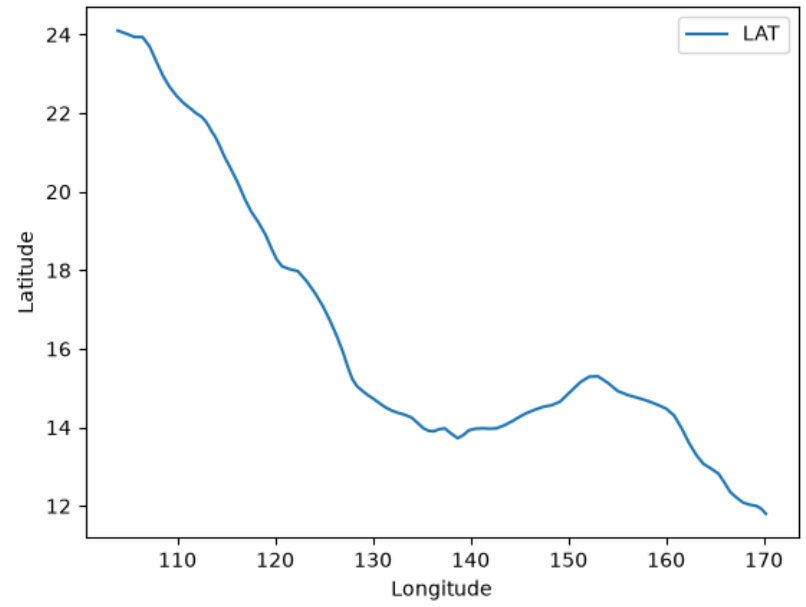
3.4 生成一个六边形密度图（hexbin plot），显示数据点在纬度和经度上的分布位置。

浏览器检索学习 Python 中 hexbin 图的绘制技巧



3.5 找到 2018 年的台风“芒克胡特”（Typhoon Mangkhut），并绘制其路径图。

利用布尔索引筛选 NAME 为 MANGKHUT 且年份为 2018 的数据，然后绘图



3.6 创建一个筛选后的数据框，仅包含自 1970 年以来来自西北太平洋（“WP”）和东海太平

洋（“EP”）流域的数据。本题目的其余部分将使用该数据框。

用布尔索引进行筛选

```
[36]: #3.6
data=df[(df['ISO_TIME'].dt.year>=1970) & ((df['BASIN']=='WP') | (df['BASIN']=='EP'))]
data
```

[36]:

	SID	SEASON	NUMBER	BASIN	SUBBASIN	NAME	ISO_TIME	NATURE	LAT	LON	WMO_WIND	WMO_PRES	WMO_AGENCY
350393	1970050N07151	1970	22	WP	MM	NANCY	1970-02-19 00:00:00	TS	7.00000	151.400		1006	tokyo
350394	1970050N07151	1970	22	WP	MM	NANCY	1970-02-19 03:00:00	TS	7.24752	151.205			
350395	1970050N07151	1970	22	WP	MM	NANCY	1970-02-19 06:00:00	TS	7.50000	151.000		1002	tokyo
350396	1970050N07151	1970	22	WP	MM	NANCY	1970-02-19 09:00:00	TS	7.75747	150.772			
350397	1970050N07151	1970	22	WP	MM	NANCY	1970-02-19 12:00:00	TS	8.00000	150.500		998	tokyo
...	...	...	...	...	...	...	...	...	...	...	...	...	...
707084	2022275N10316	2022	76	EP	MM	JULIA	2022-10-10 15:00:00	TS	13.99570	-90.294			
707085	2022275N10316	2022	76	EP	MM	JULIA	2022-10-10 18:00:00	NR	14.50000	-91.000			
707173	2022286N15151	2022	80	WP	MM	NaN	2022-10-12 12:00:00	NR	15.20000	151.300			
707174	2022286N15151	2022	80	WP	MM	NaN	2022-10-12 15:00:00	NR	15.05000	151.325			
707175	2022286N15151	2022	80	WP	MM	NaN	2022-10-12 18:00:00	NR	14.90000	151.350			

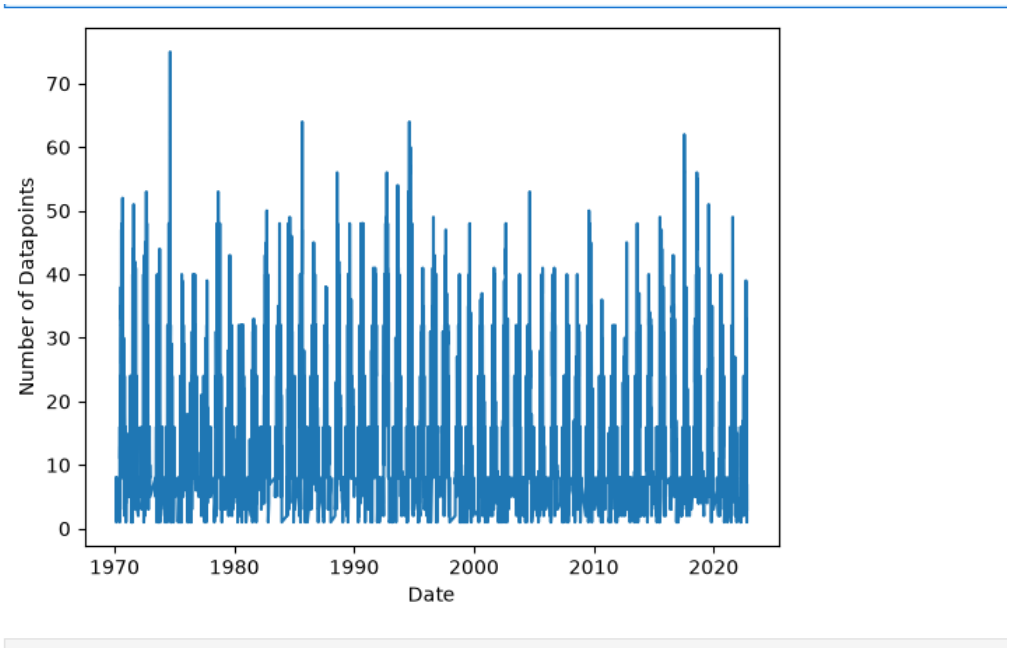
176352 rows × 17 columns



3.7 绘制每日数据点数量的分布图。

提取日期，按日期分组计数，然后画图

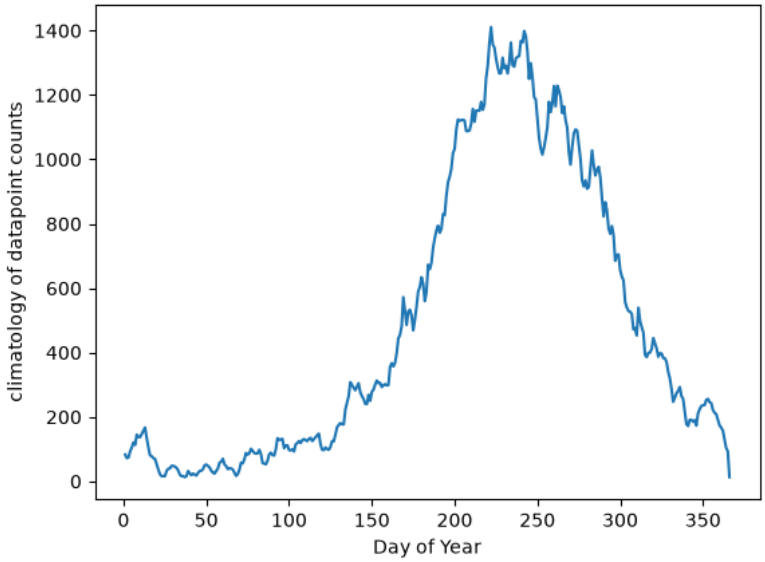
运行代码一直不显示图，问询 deepseek 提示在代码上方加入“%matplotlib inline”再重新运行。



3.8 计算每日数据点数量随年份日数变化的气候学趋势。年份日数从 1 月 1 日开始按顺序编号。

利用 dt.dayofyear 提取一年中的第几天，再用按 dayofyear 分组求平均值

我查阅了 Pandas 文档，发现使用 dt.dayofyear 可以快速将日期转化为一年中的第几天



3.9 计算每日计数与气候学趋势之间的偏差（异常值）。

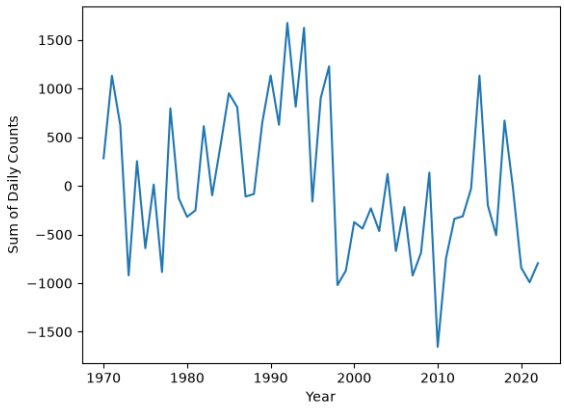
偏差=实际每日数-该日期对应的气候学趋势均值。

```
[23]: ISO_TIME
      1970-02-19    7.018868
      1970-02-20    7.094340
      1970-02-21    7.188679
      1970-02-22    7.377358
      1970-02-23    7.490566
      Freq: D, dtype: float64
```

```
[8]: #3.10
```

3.10 将异常序列按年度分辨率重新采样，并绘制图表。哪些年份表现出异常的飓风活动？

提取年份、按年分组并求和画图，找出异常突出的年份。




```
[ ]:
```

根据降序的结果，异常活跃（异常偏高）的年份 1992 年和 1994 年的异常值极其突出，远远甩开了其他年份；其次是 1997 年 和 1990 年。2015 年也相对较高。根据升序的结果，异常偏的年份 2010 年的负异常非常夸张，远远低于历史平均；其次是 1998 年、2021 年、

2007 年 和 1973 年。

```
[33]: ISO_TIME
      1992    1674.603774
      1994    1624.849057
      1997    1228.849057
      1990    1134.849057
      2015    1132.849057
      1971    1131.849057
      1985     951.849057
      1996     904.603774
      1993     813.849057
      1986     808.849057
      dtype: float64
```

```
[35]: SumCounts.sort_values().head(
```

```
[35]: ISO_TIME
      2010   -1655.150943
      1998   -1020.150943
      2021    -991.150943
      2007    -922.150943
      1973    -920.150943
      1977    -886.150943
      1999    -872.150943
      2020    -841.396226
      2022    -794.981132
      2011    -742.150943
      dtype: float64
```

4、

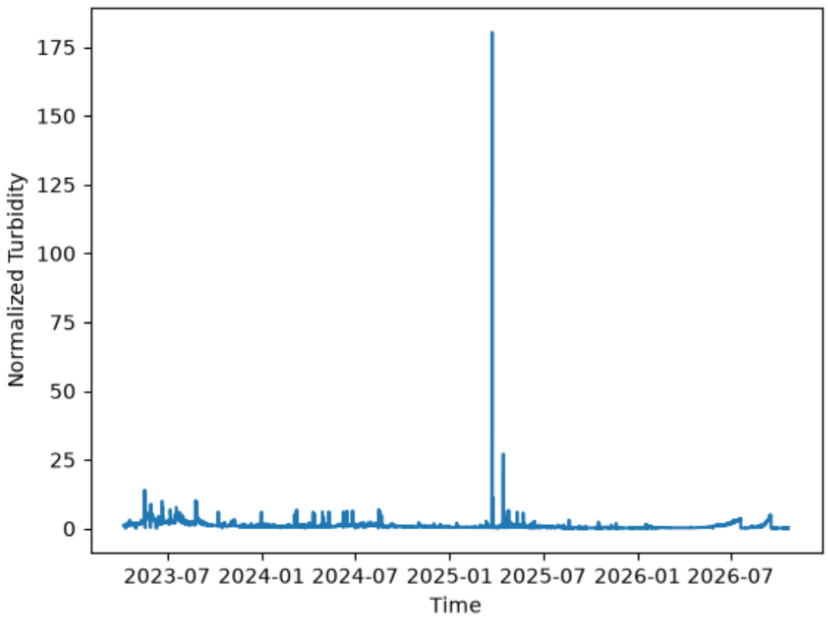
4.1 加载 csv、XLS 或 XLSX 文件，并清理可能存在的缺失值或质量不佳的数据点。

4.2 绘制某一变量的时间序列图。

选择均一化浊度绘图

[9]:
```
#4.2 选择均一化浊度

plt.plot(welldata['监测时间'], welldata['均一化浊度'])
plt.xlabel('Time')
plt.ylabel('Normalized Turbidity')
plt.show()
```



4.3 对该变量进行至少 5 项简单的统计检验，并报告你的发现。

我对数据进行了基础性描述、最大值出现时间、最小值出现时间、直方图与按小时计算平均浊度的简单的统计检验，发现该站点均一化浊度呈现极度右偏分布（均值 1.0 远大于中位数 0.6），绝大多数时间水体清澈，但受极少数极端事件影响（最高达 180.3）；同时，平均浊度存在显著的日变化周期，在下午 14:00 达到峰值后 15:00 骤降。

[5]:
```
#4.3.1基础特征
welldata['均一化浊度'].describe()
```

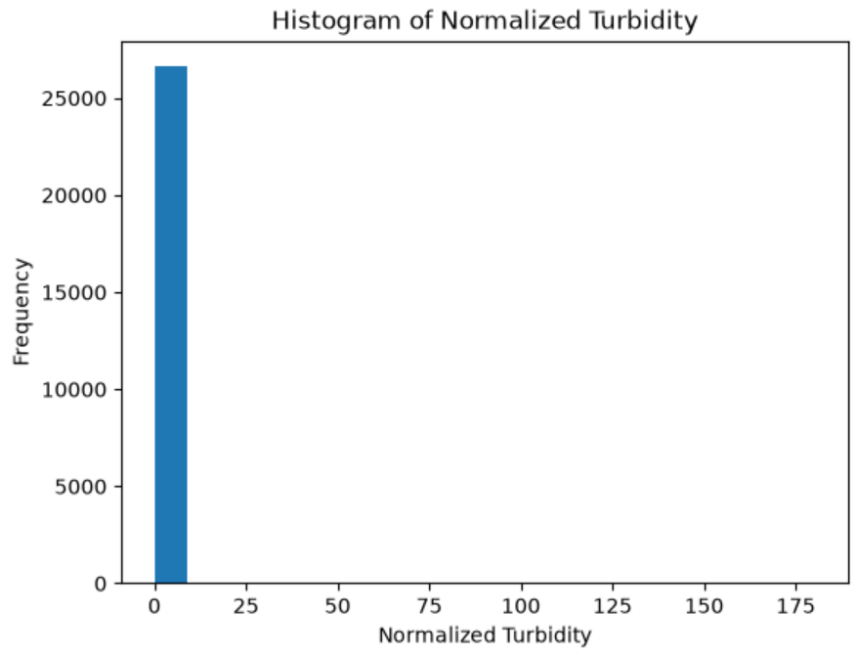
[5]:
```
count    26675.000000
mean         1.000000
std          3.754472
min          0.014567
25%          0.303527
50%          0.598105
75%          1.008397
max        180.337855
Name: 均一化浊度, dtype: float64
```

[7]:
```
#4.3.2最大值出现的时间
welldata['均一化浊度'].max()
welldata.loc[welldata['均一化浊度'].idxmax(), '监测时间
```

[7]:
```
Timestamp('2025-03-24 09:00:00')
```

10]:
```
#4.3.3最小值出现的时间
welldata['均一化浊度'].min()
welldata.loc[welldata['均一化浊度'].idxmin(), '监测时间
```

10]:
```
Timestamp('2025-10-15 16:00:00')
```



[22]:
```
#4.3.5按小时计算平均浊度
welldata['Hour'] = welldata['监测时间'].dt.hour
welldata.groupby('Hour')['均一化浊度'].mean().plot()
plt.xlabel('Hour of Day')
plt.ylabel('Mean Normalized Turbidity')
plt.show()
```

